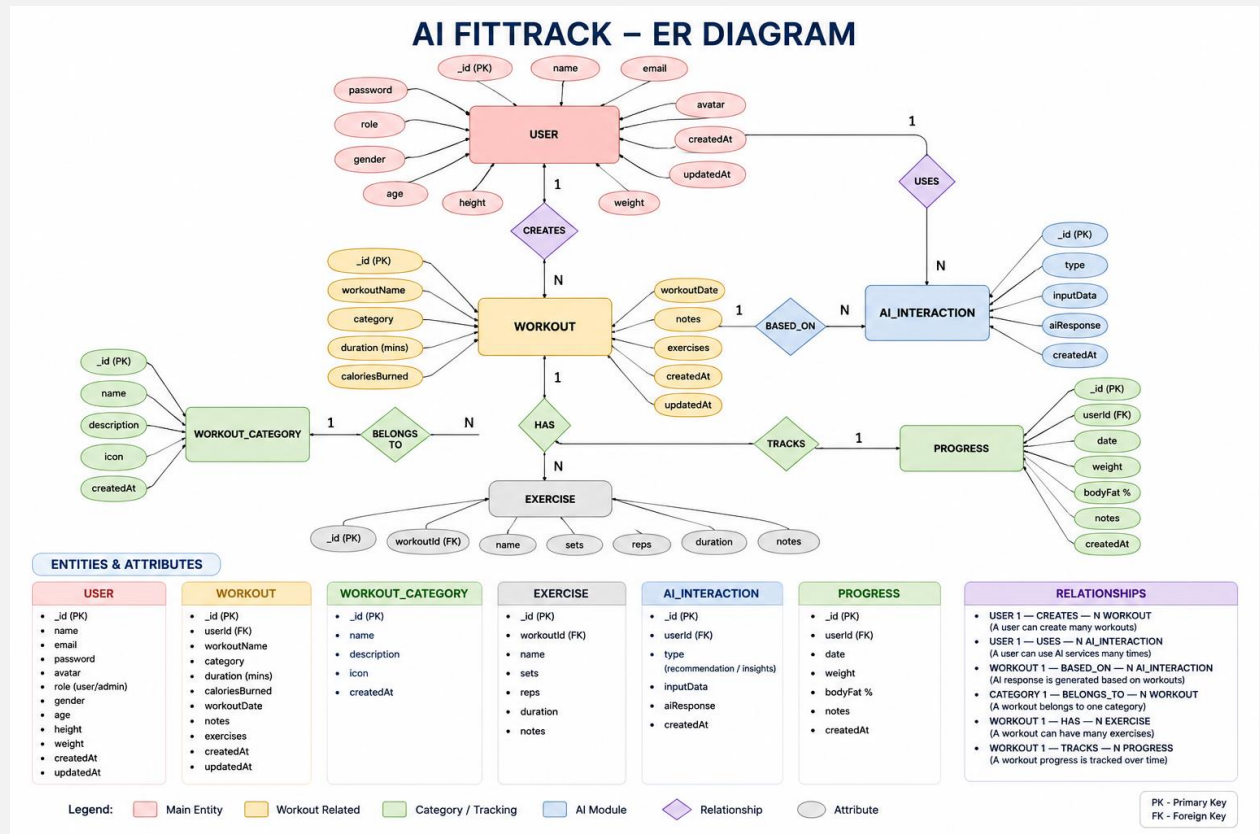




Project Planning Phase

Entity Relationship (ER) Diagram Description

The AI FitTrack backend consists of two primary collections that are logically connected through a one-to-many relationship.



1. User Entity

The User entity stores authentication and profile information.

Attributes

Field	Type	Description
_id	ObjectId	Primary Key
name	String	User Name
email	String	Unique Email Address
password	String	Encrypted Password
createdAt	Date	Record Creation Time

updatedAt	Date	Last Updated Time
-----------	------	-------------------

Constraints

- Email must be unique.
- Password is encrypted using bcrypt.
- Name is mandatory.
- JWT token is generated after successful login.

2. Workout Entity

The Workout entity stores workout-related information for each authenticated user.

Attributes

Field	Type	Description
_id	ObjectId	Primary Key
user	ObjectId	References User Collection
workoutName	String	Workout Name
category	String	Workout Category
duration	Number	Duration (Minutes)
caloriesBurned	Number	Calories Burned
workoutDate	Date	Workout Date
createdAt	Date	Created Timestamp
updatedAt	Date	Updated Timestamp

Relationship

User → Workout

Relationship Type: One-to-Many

A single user can create multiple workout records. Each workout belongs to only one authenticated user.

Example:

John Doe

- Morning Running
- Evening Jogging
- Upper Body Strength
- Cycling Session

This relationship is maintained using the **user** field inside the Workout schema, which stores the ObjectId of the corresponding user.

Key Features

The FitTrack AI provides a complete backend solution for workout tracking and AI-assisted fitness guidance.

1. Secure Authentication

The application implements JWT-based authentication.

Features include:

- User Registration
- Secure Login
- Password Encryption
- JWT Generation
- Protected APIs

2. Workout Management

Users can perform complete CRUD operations.

Supported Operations

- Add Workout
- View Workout
- Update Workout
- Delete Workout

Each workout is linked to its authenticated user.

3. Workout Search

Users can search workouts using:

- Workout Name
- Workout Category
- Workout Date

This improves accessibility of workout history.

4. Google Gemini Integration

The backend communicates with Google Gemini AI to generate intelligent workout recommendations.

Input

- Age
- Fitness Goal
- Experience Level

Output

- Weekly Workout Plan
- Suggested Exercises
- Training Tips
- Safety Recommendations

5. AI Fitness Insights

The AI module analyzes workout statistics.

Input

- Total Workouts
- Average Duration
- Calories Burned

Output

- Performance Analysis
- Improvement Suggestions
- Motivational Advice
- Fitness Progress Summary

6. Centralized Error Handling

The application returns meaningful JSON error responses for:

- Invalid Credentials
- Missing Authentication Token
- Validation Errors
- Database Errors
- Internal Server Errors

7. RESTful API Design

The backend follows REST principles.

Supported HTTP Methods

- GET
- POST
- PUT
- DELETE

Responses are returned in JSON format.

8. Modular Architecture

The project separates responsibilities into

- Models
- Controllers
- Routes
- Middleware
- Services
- Configuration

This improves maintainability and scalability.

Roles and Responsibilities

The AI FitTrack backend supports one primary role.

Authenticated User

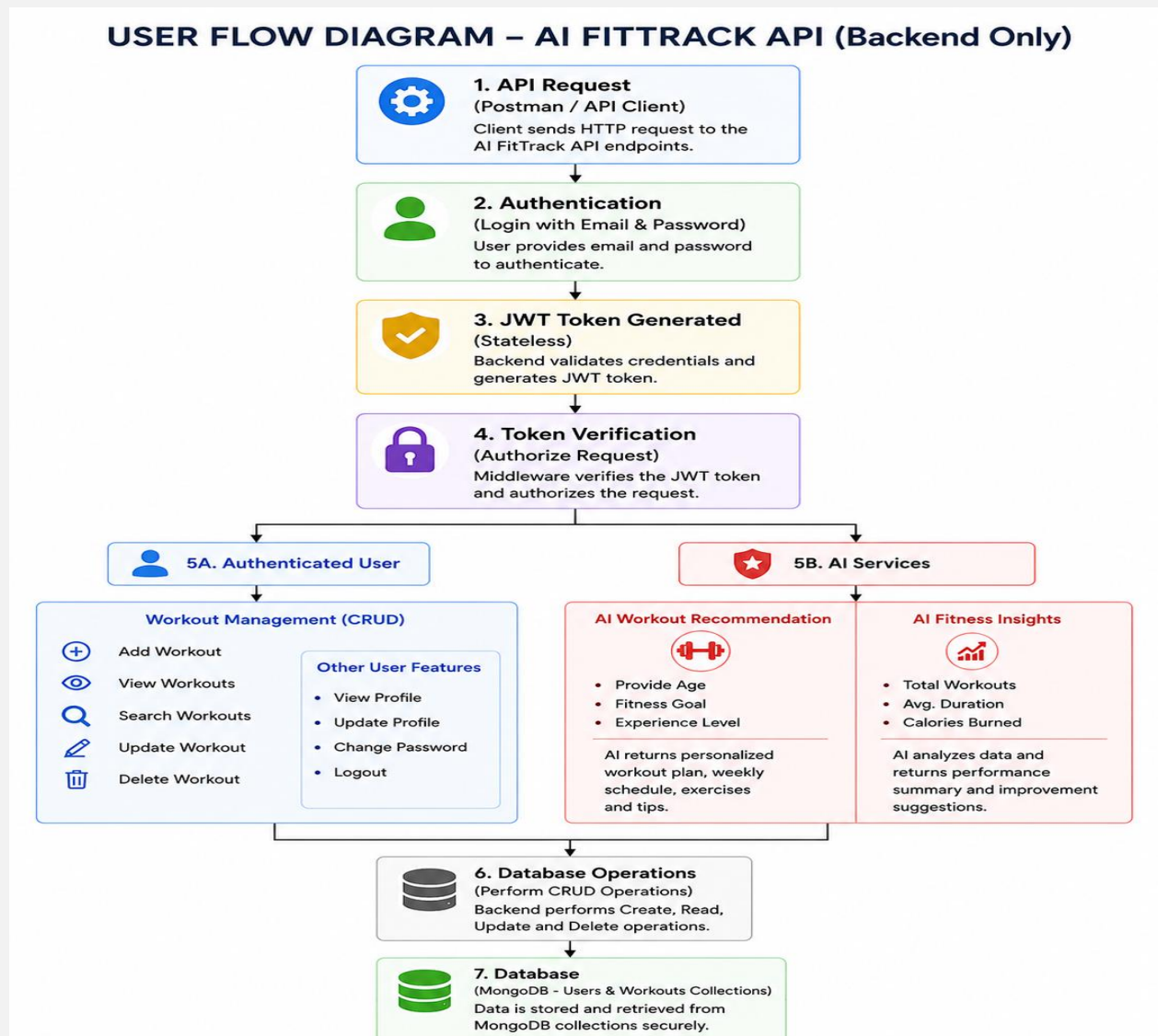
Responsibilities

- Register an account.
- Login securely.
- View profile information.
- Add workout records.
- View workout history.
- Search workouts.
- Update workouts.
- Delete workouts.
- Request AI workout recommendations.
- Request AI fitness insights.

Permissions

Operation	Permission
Register	Yes
Login	Yes
View Profile	Yes
Add Workout	Yes
Get Workouts	Yes
Search Workouts	Yes
Update Workout	Yes
Delete Workout	Yes
AI Recommendation	Yes
Fitness Insights	Yes

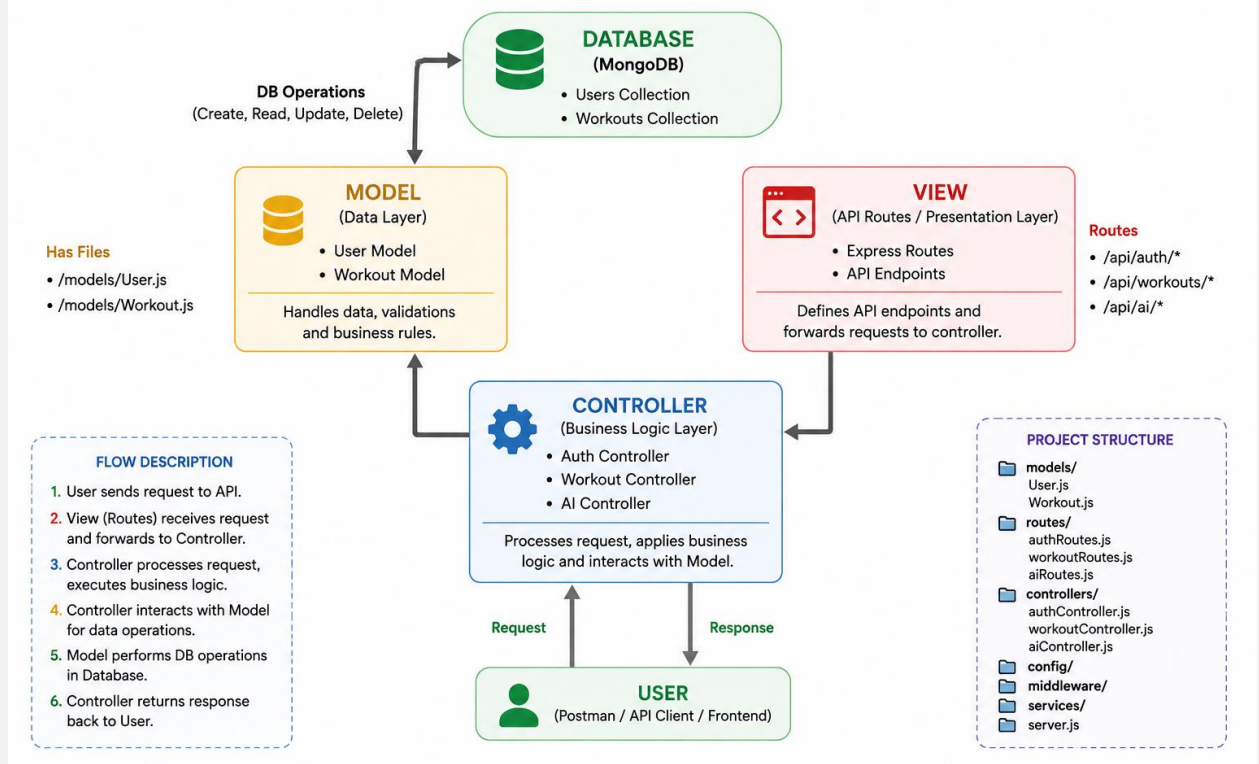
User Flow



MVC Architecture Pattern

The FitTrack AI is designed using the **Model-View-Controller (MVC)** architecture pattern. This architecture separates application logic into independent components, improving code organization, maintainability, and scalability.

AI FITTRACK API (BACKEND) – MVC PATTERN



Model Layer (Data Layer)

The Model Layer defines the structure of the application's data using Mongoose schemas. It communicates directly with MongoDB and performs database operations.

Models in the project include:

- User Model
- Workout Model

Responsibilities:

- Schema Definition
- Data Validation
- CRUD Operations
- Collection Mapping

View Layer (API Interface)

Since AI FitTrack is a backend REST API, there are no traditional UI templates. The View Layer is represented by API consumers such as React applications, mobile applications, or Postman, which interact with the backend through HTTP requests and receive JSON responses.

Responsibilities:

- Send Requests
- Receive JSON Responses
- Display Data
- Handle Client-side Interaction

Controller Layer (Business Logic)

The Controller Layer acts as the intermediary between the View and Model. It processes incoming requests, validates input, invokes services and models, and sends structured JSON responses back to the client.

Responsibilities:

- Request Validation
- Business Logic Execution
- Database Interaction
- AI Service Invocation
- Response Generation

The separation of these three layers ensures loose coupling, simplifies testing, enhances code readability, and makes the application easier to extend with future features.